

Análisis de datos

Carrera de Especialización en
Inteligencia Artificial
B4 2025



Programa de la materia

1 **Introducción al análisis de datos**
Flujo de trabajo típico. Aplicaciones. Configuración del entorno. Introducción a las principales bibliotecas.
28/8

2 **Análisis exploratorio de datos (EDA)**
Tipos de variables. EDA y visualización de variables numéricas y categóricas. Identificación de valores faltantes y outliers. Relación entre variables.
4/9

3 **Taller práctico - parte 1**
Trabajo en grupos: EDA y visualización inicial de un dataset. Buenas prácticas. Puesta en común.
11/9

4 **Preprocesamiento y limpieza de datos**
Flujo de preparación de datos. Prevención de data leakage. Tratamiento de datos faltantes y outliers. Cardinalidad. Codificación. Discretización.
18/9

5 **Procesamiento (continuación)**
Normalización y estandarización. Técnicas avanzadas para el tratamiento de datos faltantes y outliers. Desbalance. Creación de nuevos features.
25/9

6 **Reducción de dimensionalidad**
Métodos de extracción y técnicas para selección de features. **Bonus:** EDA de datos no estructurados.
2/10

7 **Taller práctico - parte 2**
Trabajo en grupo: preprocesamiento y feature engineering de un dataset. Buenas prácticas. Puesta en común. Consultas.
9/10



13/10 - límite para compartir el material para la exposición final.

8 **Exposición final**
Exposición de trabajos finales.
16/10

Bonus: Herramientas para automatizar el EDA y otras tareas del flujo de trabajo.

→ Normalización y estandarización

- La normalización y la estandarización son dos técnicas para preparar las variables con el objetivo de mejorar el rendimiento de los modelos.

- **Normalización.** Se escalan las variables numéricas para que sus valores estén dentro de un rango específico, generalmente entre 0 y 1.

- Útil cuando se usa un algoritmo que depende de la distancia (ej. KNN)

En KNN, si no se escalan los datos, el algoritmo puede darle mas importancia a los valores mas grandes

$$X_{\text{normalizado}} = \frac{X - \min(X)}{\max(X) - \min(X)}$$

- **Estandarización** (o Z-score). Se transforman los datos de modo que tengan una media igual a 0 y una desviación estándar igual a 1. Adecuada cuando los datos ya tienen una distribución normal.

- Útil para algoritmos que asumen distribución normal (Regresión lineal, SVM)

$$X_{\text{estandarizado}} = \frac{X - \mu}{\sigma}$$

Actividad



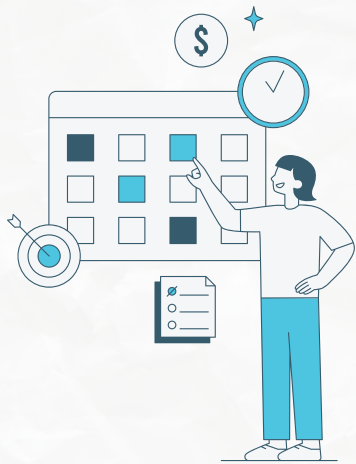
Escalamiento después de codificación

- La decisión de escalar las variables categóricas dependerá del algoritmo de ML a usar y del mecanismo de codificación empleado.
- Excepto algunos casos, la mayoría de las técnicas pueden producir rangos grandes de números y es una buena práctica escalar siempre después de codificar (especialmente si se utilizarán algoritmos basados en distancia).
- En general, no se escalan los features binarios (one-hot encoded) ya que esto no aporta ningún valor (y en algunos casos puede incluso ser contraproducente).
- Tampoco es aconsejable modificar features codificados con cyclic encoding, ya que se "rompe" el efecto cíclico.

No importa si son variables numericas o categoricas, siempre se quiere aplicar un escalamiento para asegurar que todos los datos esten en una misma escala



→ Técnicas avanzadas para el tratamiento de datos faltantes



Habíamos visto métodos simples como:

- Eliminación de filas (observaciones) y columnas (variables).
- Imputación por métodos univariados (ejemplo: imputar por la media)

Además, podemos aplicar imputación por métodos multivariados (a partir de valores de otras columnas):

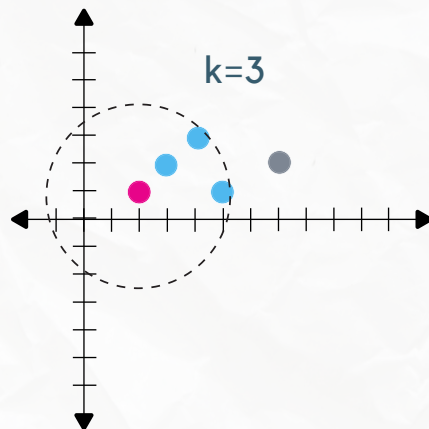
- modelos estadísticos (ej., MICE)
- algoritmos de ML (por ej., regresión, árboles de decisión, KNN).

Métodos multivariados - KNN

KNN (K-Nearest Neighbors):
busca los "k" vecinos que se
parecen más a la fila del dato
faltante. Este se calcula luego a
partir de esos vecinos.

Atención:

- Se recomienda normalizar los datos
antes de usar este algoritmo porque es
sensible a la escala.
- Puede ser ineficiente en grandes
volúmenes de datos.



v1	v2	v3
4	3	2
7	2	2
2	1	NaN
5	1	4
3	2	3

En este caso, el valor
faltante se calculo con
el promedio de los k
vecinos.

KNNImputer

Gallery examples: Imputing missing values before building an estimator
Release Highlights for scikit-learn 0.22

scikit-learn

Métodos multivariados - MICE

- MICE (Multiple Imputation by Chained Equations) es una técnica avanzada que predice valores faltantes por medio de modelar cada variable incompleta usando las otras columnas.
- Es una de las técnicas de imputación multivariada más utilizada.
- Consiste en calcular valores faltantes para cada columna como una regresión lineal de las restantes (se pueden usar otros modelos).
- Es un proceso iterativo.



MICE - Funcionamiento



- Comienza con una imputación rápida y simple (Ej., la media).
- Iteración: Para cada columna con NaNs, predice los valores en base a las otras variables.
- Hace varias iteraciones, hasta que los valores predichos quedan estables.

Mice imputation (statsmodels).

IterativeImputer

Gallery examples: Imputing missing values with variants of IterativeImputer Imputing missing values before building an estimator



(Parecido a MICE, no es igual)

Dataset con nulos

A	B	C

1 Imputación simple

A	B	C
		Media C
Media A		
	Media B	

MICE paso a paso



Ojo! la notación $A \sim B + C$ se llama "fórmula estilo Patsy" y significa que A se predice a partir de B y de C (no es una suma!).

2 Ajustamos un modelo $A \sim B + C$

A	B	C
		Media C

3 Calculamos los nulos A_i con el modelo y los B_i, C_i

A	B	C
		Media C
Pred A_i	B_i	C_i

4 Ajustamos un modelo $B \sim A + C$ y calculamos nulos de B

A	B	C
		Media C
Pred A		

5 Ajustamos un modelo $C \sim A + B$ y calculamos nulos de C

A	B	C
Pred A		

6 Iteración 1 completa!

A	B	C
		Pred C 1
Pred A 1		
	Pred B 1	

7

A	B	C
		Pred C 1
	Pred B 1	

8

A	B	C
		Pred C 1
Pred A 1		

9

A	B	C
Pred A 1		
	Pred B 1	

10

A	B	C
		Pred C 1
Pred A 1		
	Pred B 1	

Estrategias avanzadas para tratamiento de faltantes

Si son MCAR y son pocos datos, se pueden eliminar esos datos y el dataset no va a quedar sesgado porque la falta era aleatoria.

- Si son MAR con patrones claros y el porcentaje es moderado a alto (20-50%):
 - **Imputación con modelos de ML** para variables críticas.
 - **Algoritmos como Random Forest Imputer o KNN-imputer** si los patrones son no lineales.
- Si se sospecha MAR^{o MNAR} con patrones complejos (los faltantes dependen de muchas variables):
 - **Multiple Imputation by Chained Equations (MICE).**

No se pueden eliminar esos datos porque estaría sesgando los datos.

Ejemplos prácticos en Jupyter



→ Técnicas avanzadas para el tratamiento de outliers



Habíamos visto métodos simples como:

- Eliminación de filas (observaciones) *Si son pocos, se pueden eliminar*
- Imputación por métodos univariados (ejemplo: imputar por la media, *capping*)
- Minimizar el impacto con transformaciones (ejemplo: logaritmo) *No se eliminan outliers, se minimiza el impacto*
- Segmentación *Ejemplos: "datos buenos" y "datos malos" (no se eliminan outliers)*

Además, podemos aplicar imputación por métodos multivariados (a partir de valores de otras columnas):

- algoritmos de ML (por ej. KNN).



A diferencia de los datos faltantes, las técnicas avanzadas no saben cuáles son los outliers. Por lo tanto, lo que se hace es eliminar los outliers (poner nulos) y luego se tratan como faltantes.

Ejemplos prácticos en Jupyter



→ Creación de nuevos features



- Se trata de generar nuevas variables a partir de otras existentes.
- El objetivo es captar patrones o relaciones ocultas y mejorar la interpretación.

Ejemplo:

- $IMC = \text{peso} / (\text{altura})^2$
- Reemplazar las variables peso y altura por la métrica IMC, que está más relacionada con la salud que las originales.

Criterios para crear features

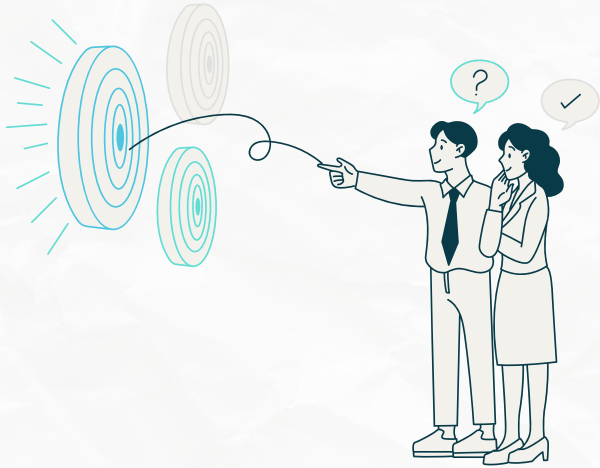


- Deben que ser relevantes para el objetivo.
- Tienen que capturar relaciones importantes o transformaciones de variables guiadas por conocimiento del dominio o análisis.
- Tienen que complementar o superar lo que aportan las variables originales.
- Creados teniendo en cuenta los modelos que los consumirán.
- Hay un balance entre costo computacional e impacto en performance.

Ejemplos de casos de uso:

- Dataset con pocas filas.
- Patrones no lineales.
- El conocimiento del dominio lo pide (deuda/ingreso, nro. de ítems por compra, etc).

(Algunas) técnicas para crear features



- Matemáticas: A/B , $A*B$, $A-B$, etc.
- Agregados: promedios, totales por grupo, sumas, etc.
- Temporales: extraer día, hora, mes, de un timestamp.
- Texto: TF-IDF (importancia de una palabra), generación de embeddings.

Ejemplos comunes

- Fecha de ingreso → Antigüedad/categoría (senior, semi-senior, etc.)/ tiempo desde el último evento (por ej promoción).
- Fecha de nacimiento → Edad.
- Timestamp → Hora del día/Es día hábil?/fin de semana?/feriado?/llovió ese día?/Temporada?
- Factores de riesgo (tabaquismo, hipertensión, diabetes, etc.) → Conteo /Nivel de riesgo ponderado/ocurrencia simultánea.
- Valores faltantes → Flag / conteo de faltantes por fila / faltante por dominio (ej: financiero, demográfico, etc.)
- Número de cel → localización geográfica a través del código de área.
- Sueldo/provincia/profesión → Salario promedio por provincia por profesión (ojo! media global si hay combinaciones muy raras)
- Transacciones/usuarios → número de transacciones por usuario.
- *Feedback* de un cliente → *Sentiment score* (positivo, neutral, negativo)
- Descripción de un producto → Palabras clave (descuento, oferta, premium, etc.)

Tip: conocer los modelos

- Agregar features con relaciones complejas (ej., IMC) cuando se trabaja con modelos lineales.
- Considerar agregar ratios, que son difíciles de aprender para casi todos los modelos.
- Los árboles pueden aprender casi cualquier combinación de features, pero cuando hay alguna combinación específica que nos interesa conviene crearla explícitamente (especialmente si hay pocos datos).
- Los features de tipo aggregation son especialmente útiles para modelos de tipo árbol porque estos no tienen la capacidad natural de combinar información de muchos features a la vez.

Cuidado con el data leakage



- Crear features después del split del dataset.
- Evitar features que son casi idénticos a la variable target o que son una consecuencia directa de esta y la "revelan". Ej: predecir transacciones fraudulentas usando la variable "se inició devolución del cargo".

→ Desbalance (De la variable target para problemas de clasificacion)

Se produce cuando las clases (o etiquetas) que el modelo debe predecir no están representadas de manera proporcional en el dataset.

Causas:

- Sesgo al momento de recolectar los datos.
- Naturaleza del dominio en cuestión (causa más frecuente).

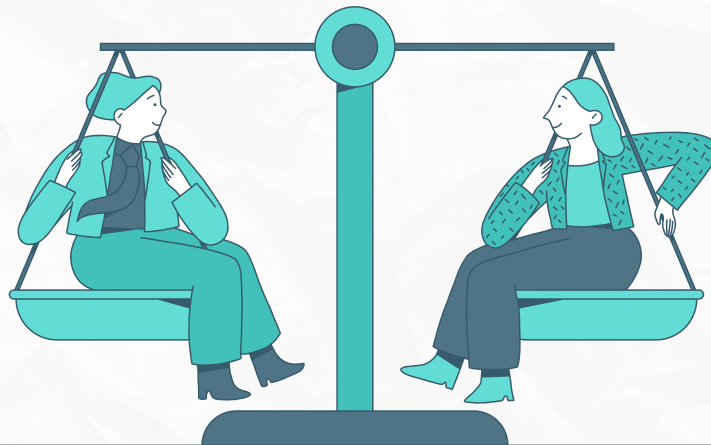


Desbalance

¿Por qué es un problema?

- Los modelos de ML tienden a aprender mejor de las clases más fuertes (mayoritarias) y tener un mal desempeño para las clases menos frecuentes (minoritarias).
- En consecuencia las métricas como precision, recall, F1-score y AUC-ROC, pueden ser engañosas.

Ejemplo: un modelo que siempre predice la clase mayoritaria tiene alta accuracy pero bajo recall en las clases minoritarias.



Técnicas para detectar desbalance

Técnica	Explicación
Análisis de la distribución	Análisis visual (gráfico de barras, pie chart) o cálculo de las proporciones de cada clase
Matriz de confusión Luego de entrenar el modelo	Según cómo se clasificaron las instancias puede revelarse un desbalanceo de clases.
Índice de Gini $G = 1 - \sum p_i^2$	Mide la pureza de los datos. Indirectamente puede reflejar si hay un desbalance entre clases. ~0.5 indica un dataset balanceado.
Entropía de Shannon $H = - \sum_{i=1}^n p_i \log_2(p_i)$	Mide la incertidumbre de los datos. Mientras más alta la entropía, mayor la incertidumbre, más balanceados los datos.

Técnicas para el tratamiento del desbalance

Técnica para <u>balancear</u>	Explicación
<i>Oversampling</i> (sobremuestreo)	Duplicar o recrear en forma sintética las muestras de la clase minoritaria: Ej. SMOTE (<i>Synthetic Minority Over-sampling Technique</i>) o IA generativa.
<i>Undersampling</i> (submuestreo)	Remover muestras de la clase mayoritaria.

Técnica para <u>mitigar</u>	Explicación
<i>Weighted Random Sampler</i> (muestreo aleatorio ponderado)	Utiliza muestreo con reemplazo proporcional a los pesos (generalmente inversos a la frecuencia de la clase). Esto simula un dataset equilibrado sin alterar los datos originales.
<i>Weighted Loss Functions</i> (Funciones de pérdida ponderadas)	Asigna pesos a las clases en la función de pérdida (ej. en <i>Cross-Entropy Loss</i>). Esto hace que el optimizador (ej. Adam) se enfoque más en minimizar errores en clases minoritarias.
Técnicas de ensamble	Algoritmos como <i>Balanced Random Forest</i> y AdaBoost tienen mecanismos internos para manejar el desbalance.

En desbalances extremos (1:1000), considerar recolectar más datos. En este caso no aplica over/under sampling porque puede generar problemas al inventar/sacar datos.

Importante!

Las técnicas de balanceo de clases solo se aplican a los datos de entrenamiento (validación y test deben reflejar el escenario real)

En un desbalance del 60/40 % no tiene mucho sentido aplicar alguna tecnica.
Pero valores del tipo 80/20 % si es mas conveniente, y si es aun mayor, es mejor recolectar datos.

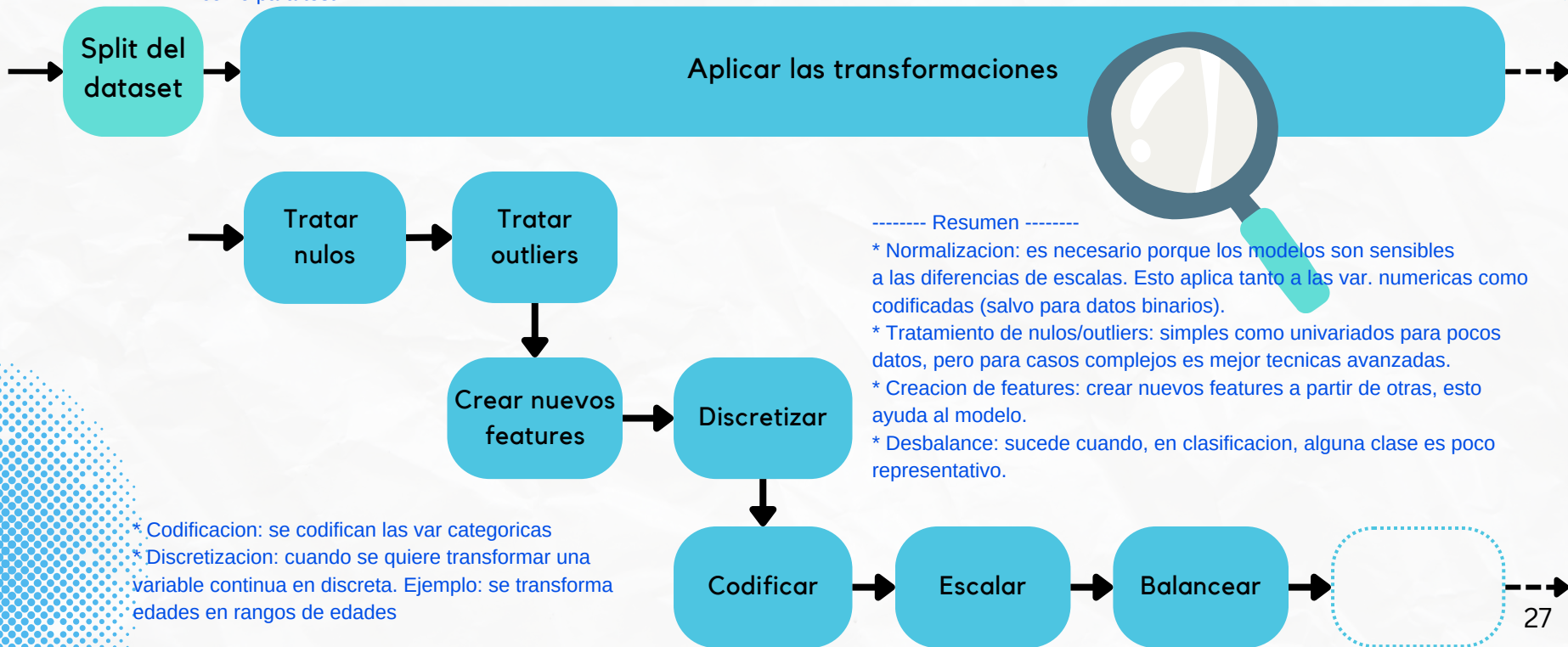


Ejemplo práctico

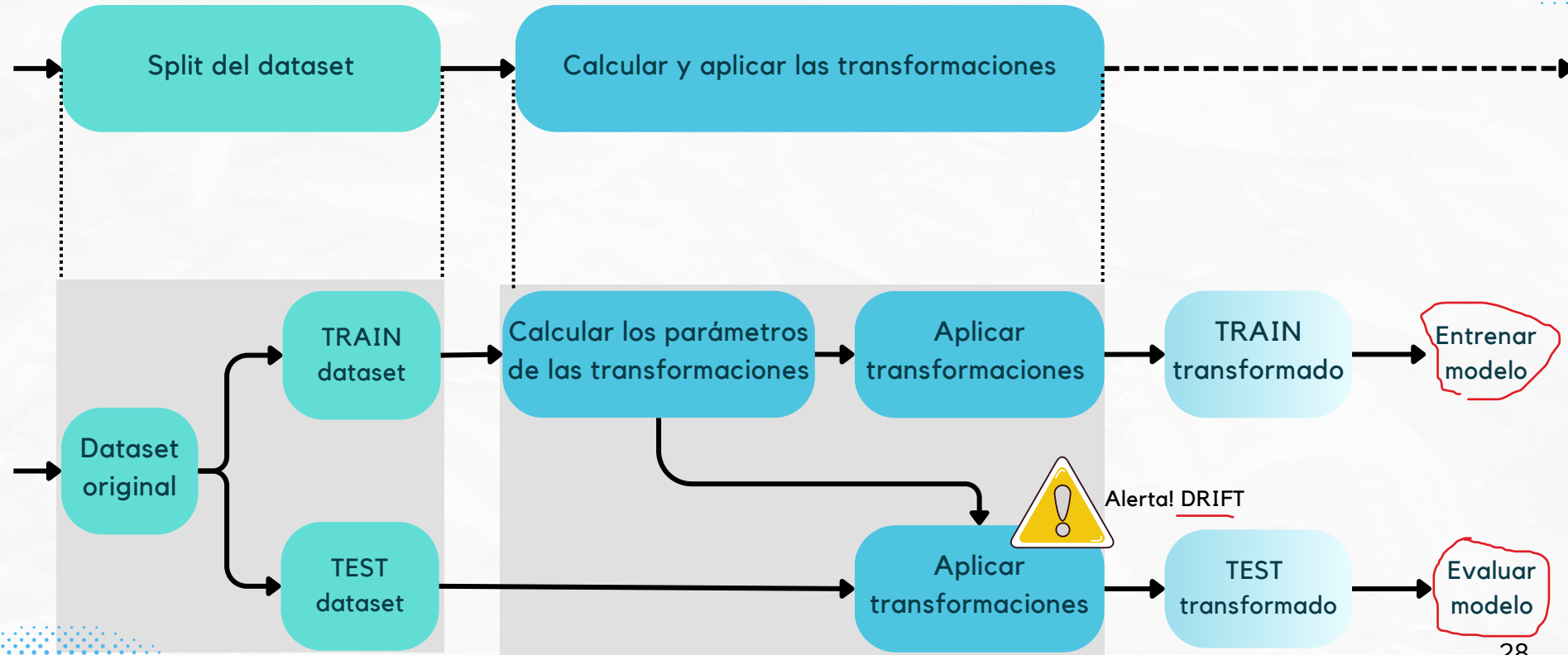


→ Flujo de preparación de datos para IA

El flujo depende del caso, pero a continuación se muestra algo típico. Recordar que el tratamiento se hace tanto para entrenamiento como para test



Flujo de preparación - Train vs. Test





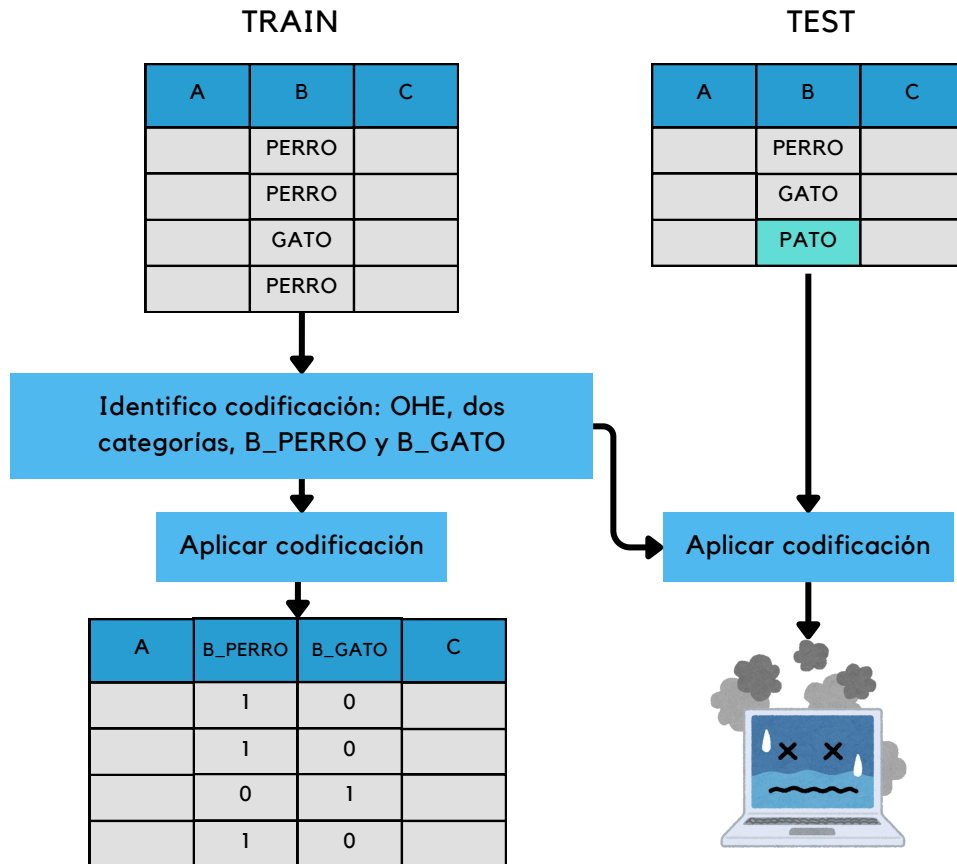
Problemas comunes - Data drift

Esto sucede cuando el dataset de train y test no tienen la misma información.

- Ajustar los parámetros de imputación, codificación, escalamiento, etc. sobre los datos de train y luego aplicarlos a train y test puede traer algunos problemas.
- Ejemplo 1 - Codificación: para una determinada variable, en el dataset de Test queda una categoría que no está en el dataset de Train.
- Ejemplo 2 - Escalamiento: en el dataset de Test una cierta variable toma un valor más grande que el máximo (y/o un valor más chico que el mínimo) de esa variable en Train.



Ejemplo 1: codificación



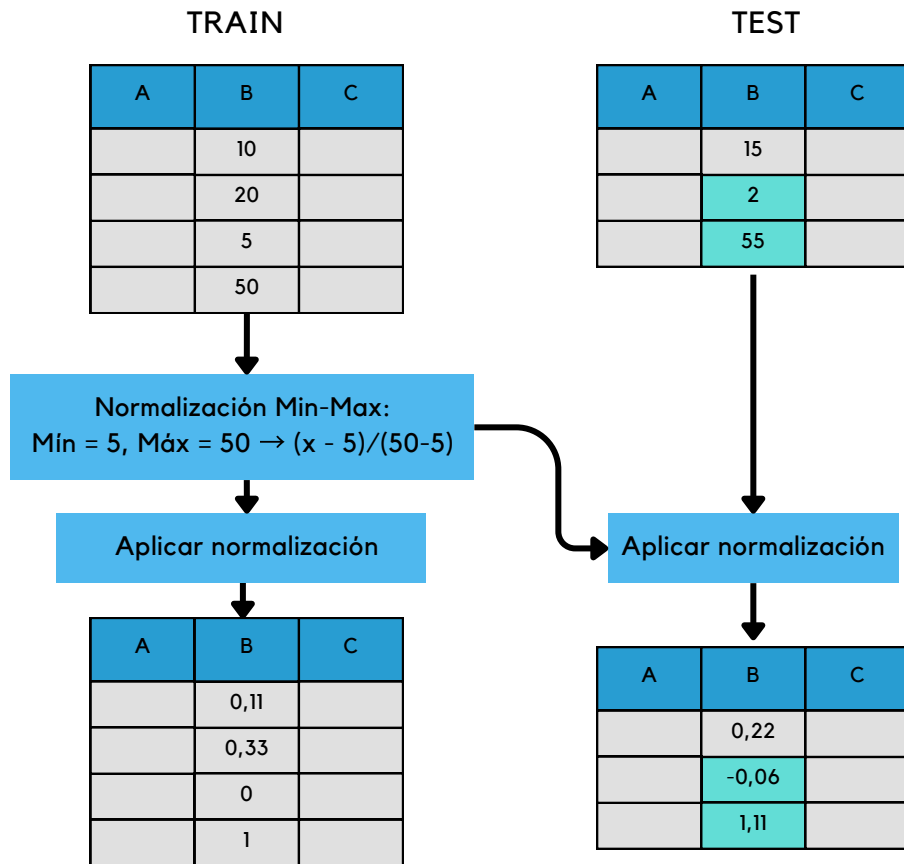
Posibles soluciones:

- Ignorar la categoría desconocida: La librería de Sklearn permite ignorar la categoría

A	B_PERRO	B_GATO	C
	1	0	
	0	1	
	0	0	

- Re-analizar los datos/el problema.

Ejemplo 2: escalamiento



- En la mayoría de los casos, esto no es un problema.
- Se puede hacer un "clipping" (forzar los valores a que caigan dentro del intervalo).

La librería de Sklearn permite hacer el "clipping"

¿Te quedaron dudas?



- Recordá que podés consultarnos durante la clase preniendo la cámara y hablando, o simplemente escribiendo en el chat.
- También podés escribirnos por correo las veces que sea necesario.



Esp. Lic. María Carina Roldán
macroldan@fi.uba.ar



Esp. Ing. Ariadna Garmendia
arigarmendia@gmail.com

Y si tenés un feedback, un pedido, una sugerencia, etc. y no te animás a decirlo, podés dejarlo por escrito en la encuesta voluntaria y anónima (disponible durante todo el bimestre).